Assignment 2
Cluster and Cloud Computing COMP90024
Social media Sentiment Analysis: Australian Elections
Team 23

David Stiles Rosselli (1750113) Chia-Chi Chen (1838704)
Mila Yoselyn Pacompia Machaca (1660172) Aakanksha Satish Bagal (1822880)
Aaditya Pathak (1703474)

May 20, 2026

Abstract

The exponential growth of social media platforms has created an unprecedented volume of publicly accessible data, offering researchers and analysts a real-time window into public sentiment and discourse. However, harvesting and deriving meaningful insight from this data at scale demands both sophisticated infrastructure and carefully designed analytical pipelines. This paper presents the design and implementation of a cloud-native platform built on the Melbourne Research Cloud (MRC), NeCTAR, developed to continuously harvest and analyse Australia-focused social media data across multiple platforms. The system leverages Kubernetes as its core orchestration layer, with infrastructure provisioned declaratively through Terraform to ensure reproducibility.

Data is continuously harvested from Mastodon, BlueSky, YouTube, and the GDELT, a news event dataset, using a serverless architecture built on Fission. After which lightweight harvester functions feed raw records into a Valkey message queue. This queue-driven approach decouples ingestion from processing, allowing the system to absorb spikes in data volume while maintaining throughput. A dedicated machine learning inference service, built on ONNX-optimised transformer models, enriches each record with sentiment scores and semantic text embeddings before it is indexed into ElasticSearch. Several analytical scenarios are explored, all connected to Australia, examining how public sentiment on social media evolves around topics such as political figures, transport, and major current events, and how these compare across platforms and against official census and government data. This paper discusses the architectural trade-offs, scalability properties, and practical challenges encountered when deploying and operating such a system on the NeCTAR Research Cloud, contributing insights into the

effective application of cloud infrastructure and big data tooling for real-world social media analytics.

Keywords: ElasticSearch, Fission, Valkey, Processor, Sentiment Analysis, NeCTAR.

- **Code Repository**: `https://gitlab.unimelb.edu.au/mpacompiamac/comp90024_team_23`
- **Video Demonstration**: `https://www.youtube.com/watch?v=_PTk-6_0nWQ`

Contents

1 Introduction **5**

2 System Architecture Overview **6**
2.1 Data Ingestion . . . 6
2.2 Data Processing . . . 6
2.3 Machine Learning Inference . . . 7
2.4 Storage and Analytics . . . 7
2.5 API and Frontend . . . 7

3 Infrastructure **7**
3.1 Infrastructure Management . . . 7
3.2 Core Infrastructure . . . 8
3.3 Deployment Procedure . . . 8
3.4 Fission Setup . . . 9
3.4.1 Environment . . . 9

4 Data Gathering **9**
4.1 Social Media Data Gathering Strategy . . . 9
4.1.1 Architecture . . . 9
4.1.2 Implementation Notes . . . 10
4.2 Social Media Providers . . . 11
4.2.1 Mastodon . . . 11
4.2.2 Bluesky . . . 11
4.2.3 Youtube . . . 13
4.2.4 Threads . . . 14
4.3 Additional Data Sources . . . 14
4.3.1 GDELT . . . 14
4.3.2 Official Data . . . 15
4.4 Data Collection Challenges . . . 16

5 Data Processing **16**
5.1 Architecture . . . 16
5.2 Machine Learning Inference . . . 17
5.3 Language Processing Limitations . . . 18
5.4 Common Index Design . . . 18
5.5 Error Handling and Safeguards . . . 19

6 Technical Discussion **19**
6.1 Extensibility . . . 19
6.2 Scalability . . . 20
6.2.1 Components . . . 20
6.2.2 Experimental Results . . . 20
6.2.3 Fault Tolerance and Single Points of Failure . . . 21
6.2.4 Testing . . . 21
6.3 Tech Stack Discussion . . . 21
6.3.1 NeCTAR . . . 21
6.3.2 Kubernetes . . . 22
6.3.3 Fission . . . 22

6.3.4 ElasticSearch 22

7 Results and Analysis **23**

8 Conclusion & Limitations **26**

1 Introduction

Social media platforms contribute to vast amounts of raw and unprocessed data, and advancements in technologies such as machine learning and artificial intelligence have made it possible to transform this data into valuable insights for businesses and researchers alike [Sivarajah et al., 2017]. Australia alone has approximately 21 million social media users [Fiber, 2026], while the number of active users worldwide exceeds 3 billion. These platforms provide a rich source of information that can be used to analyse public opinion and identify broader social and political trends.

Social media also plays a significant role in shaping how individuals perceive events, form opinions, and engage with political discourse. In this paper, we investigate public sentiment surrounding the 2025 Australian federal elections and Australian political sentiment by analysing trends in social media discussions. We aim to understand if using text analysis methods we can understand public attitudes before, during and after the elections, and identify significant political events. However, this large influx of social media data requires a scalable and efficient infrastructure for storage, indexing, and querying relevant content. Our solution aims to build such an infrastructure to enable the analysis of political trends data at scale.

In this study, we gathered data from multiple platforms including Bluesky, Mastodon, YouTube and GDELT Project. We focused on text-based data, such as social media posts or video comments. We conducted both emotion detection and sentiment analysis on the texts using open source models in order to enrich the data for analysis. While sentiment analysis broadly categorises text as positive, negative, or neutral, emotion classification captures more nuanced affective states such as anger, fear, joy, sadness, anticipation, and optimism. In the context of political discourse, these distinctions are particularly meaningful: a post may be classified as broadly negative while being primarily driven by fear rather than anger, which carries very different implications for how the public is responding to a candidate or political event. For this reason, our system performs both sentiment and emotion analysis on every harvested post, enabling a richer characterisation of public opinion over time.

The volume and velocity of social media data also introduce significant engineering challenges. Traditional data processing architectures are ill-suited to the continuous, real-time nature of social media streams, where data arrives unpredictably and in varying volumes. To address this, we adopt a cloud-native, event-driven architecture deployed on the Melbourne Research Cloud (MRC), NeCTAR, leveraging Kubernetes for container orchestration, Fission for serverless function execution, and Valkey as a message queue to decouple data ingestion from processing. This architecture allows the system to scale dynamically with data volume and ensures that spikes in social media activity do not result in data loss or processing failures.

The choice of data sources was also guided by the goal of capturing a broad and representative picture of Australian political discourse. Mastodon and Bluesky represent emerging decentralised platforms with growing Australian user bases, while YouTube provides access to comment-level reactions to political content from mainstream media channels. The GDELT Project complements these social media sources by providing structured event data derived from global news coverage, allowing us to correlate social media sentiment with real-world political events reported in the press. Together, these sources offer a multi-platform perspective on public opinion that no single data source could provide alone.

2 System Architecture Overview

We will now present an overview of the system architecture we designed and implemented, before going into further detail into the data gathering and processing in the following sections. We will do a deeper dive into overall implications of our architecture at a later section (6).

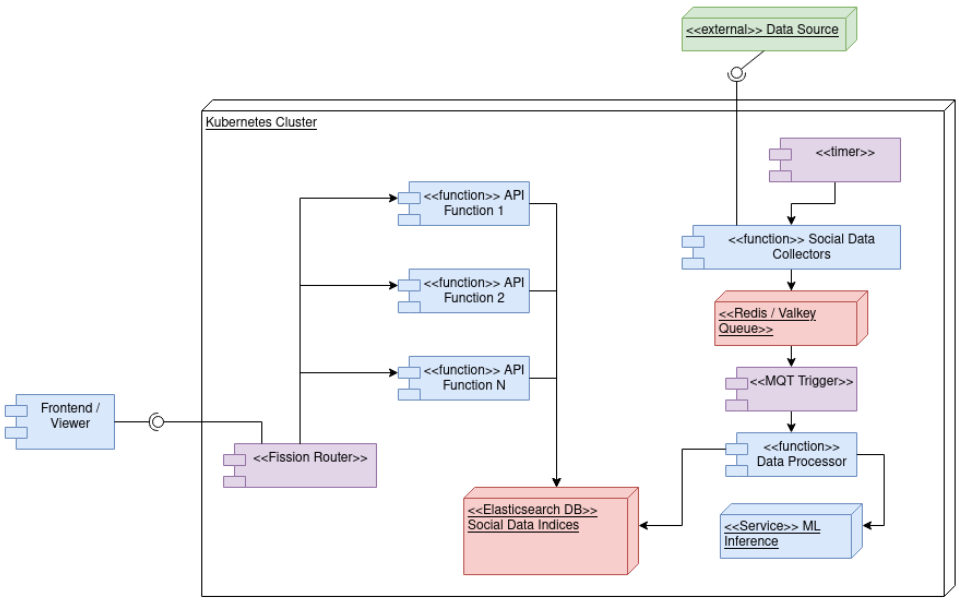


Figure 1: System Architecture Overview.

2.1 Data Ingestion

This process is handled by a collection of harvester functions deployed on Fission, an open-source Function as a Service (FaaS) framework [Fission Contributors, 2026] that runs on our Kubernetes cluster. Each data source is assigned to a separate harvester function, which is triggered by a timer setup on Fission. Instead of writing directly to the database, these harvesters publish raw records to a Valkey message queue, decoupling ingestion from processing and allowing the system to deal with sudden spikes in data volume without disrupting and overwhelming the downstream components. This design avoids the traditional Extract-Transform-Load [Vassiliadis, 2009] paradigm toward a more event driven approach, which is better suited to the continuous and unpredictable nature of social media data

2.2 Data Processing

The processing is implemented as a Fission function triggered by Kubernetes Events-Driven Autoscaling (KEDA), which is suitable for making micro-overscale with respect to the number of events that make the application alive [Molleti, 2022], whenever records are available on the Valkey queue. Each invocation of the processor receives a single raw record, which is then normalised and transformed to map it to a common schema. It then enriches it with machine learning derived features before writing it to Elastic Search. The use of KEDA allows the number of parallel processor instances to scale dynamically with queue depth, ensuring that periods of high ingestion do not result in processing backlogs.

2.3 Machine Learning Inference

To enrich the collected social media posts with sentiment scores and emotion classifications, we included a dedicated machine learning inference service deployed as a standalone Kubernetes deployment. This service exposes a simple HTTP API consumed by the processing function and runs two ONNX-optimised transformer models. Open Neural Network Exchange (ONNX) is an open-source format for artificial intelligence models, including both deep learning and traditional machine learning. It defines an extensible computational graph model, operators, and standard data types, which provides a common IR for different frameworks [Jin et al., 2020]. The first model is a sentiment analysis model trained on social media text, and the second model is a general-purpose sentence embedding model for semantic representation. Deploying this as a persistent service rather than a fission function was a deliberate decision due to the startup latency associated with loading transformer models into memory. This would have been prohibitive in a short lived serverless context. The service is stateless and can be horizontally scaled by increasing the number of replicas in its deployment configuration.

2.4 Storage and Analytics

The processed and enriched records are stored in ElasticSearch. ElasticSearch serves as both the primary data source and the analytics engine. Its document-oriented approach makes it able to efficiently search information, where schema-less data, in the form of arbitrarily complex JSON documents, can be handed to Elasticsearch for indexing [Elasticsearch B. V., 2018]. We maintain two categories of indices - provider-specific indices that preserve the raw schema of each data source for validation and source-level exploration, and a common unified index that normalises records from all providers under a single schema to enable cross-platform comparison. ElasticSearch's native support for full-text search, aggregations and vector similarity search makes it well suited to the range of analytical queries . It handled weekly sentiment aggregations and keyword-based topic filtering.

2.5 API and Frontend

Data is surfaced to end users through a Flask-based HTTP API, which exposes endpoints for querying sentiment trends, emotion distributions, candidate and party data, and census information. The API accepts topic names and associated keywords and returns pre-aggregated weekly time series data drawn from ElasticSearch. The API is implemented as Fission functions which query the ElasticSearch Database and can be called by the frontend to display analytics. The frontend itself is implemented as a Streamlit application. It uses an HTTP client to interact with our functions via the Fission router and HTTP triggers and thus pull aggregated data from ElasticSearch.

3 Infrastructure

3.1 Infrastructure Management

The overall infrastructure underpinning the platform is managed using Terraform, an open-source Infrastructure as Code (IaC) framework. The syntax is declarative with each block of code defining a resource to be provisioned. Declarative definitions (versus imperative) allow the user to define the desired state, rather than an exhaustive list of all the interim steps required to achieve that state. This approach ensures that the entire infrastructure can be version-controlled and that its reviewed and reproduced consistently across deployments. Instead of manual changes, all infrastructure definitions are expressed as Terraform configuration files.

To manage the Terraform state files, the team leveraged GitLab's built-in Terraform state backend to store files remotely on the instance instead of local files on individual machines. This ensures that all members are always working against the same view of the infrastructure, and it also enables state locking, which prevents two members from applying conflicting changes simultaneously [Valtanen, 2023]. The backend is configured with both lock and unlock addresses pointing to the GitLab Terraform state API, with a retry wait of five seconds to handle transient locking conflicts with ease.

Day-to-day infrastructure operations are further streamlined using Go-Task, a task runner that wraps common Terraform and deployment commands into named tasks defined in a `Taskfile.yml` file. This includes tasks for initialising Terraform, declared as 'task init', and validating the configuration, using 'task validate'. A few other notable tasks include 'task apply' used for applying changes and 'task deploy-resource', which deploys individual resources in isolation

3.2 Core Infrastructure

Kubernetes Cluster The cornerstone of the whole infrastructure is a Kubernetes cluster deployed on NeCTAR infrastructure using OpenStack's Magnum service for managing Kubernetes. The cluster has one control plane (or master) node and three worker nodes. All our code and backend services run on this cluster.

Storage Class Persistent storage for stateful services is provided through a custom Kubernetes storage called Perfretain. This storage class is backed by OpenStack's Cinder block storage service and is configured with a retain reclaim policy [Malla et al., 2020]. This ensures persistent data like our database files.

ElasticSearch and Kibana ElasticSearch is deployed to the Kubernetes cluster using the Elastic Operator, a Kubernetes-native operator that manage the lifecycle of ElasticSearch clusters as custom resources. The operator itself is deployed via Helm using Terraform's Helm provider. This database is then instantiated from a Kubernetes manifest that is applied by Terraform and handled by the operator. Kibana is similarly deployed using the same operator from a defined manifest. This provides a web-based interface for browsing indices, running queries and monitoring cluster health.

Fission Fission is deployed to the cluster via its official Helm chart, also managed through Terraform's Helm provider. Fission serves as the Function as a Service (FaaS) framework on which all data harvesting and processing functions run. Two additional services are deployed alongside fission to support the event-driven data pipeline

- Valkey: It is a Redis-compatible key-value store, used as the message queue that connects harvester functions to the data processing function.
- KEDA: It is deployed to enable queue-length-based autoscaling of fission

3.3 Deployment Procedure

In order to deploy, you must first supply OpenStack credentials in a *.env* file in the infrastructure repository.There are many dependencies between components of the core infrastructure, so they have to be deployed in stages. This can be done with the Taskfile utility as follows:

1. Deploy the cluster and download the kubeconfig file:
   `task deploy-resource TARGET=openstack_containerinfra_cluster_v1.main`
   Then, get the kubeconfig file for the cluster and put it at `infrastructure/config`.

2. Deploy the Elastic Operator. Deploying this at the same time as the ElasticSearch cluster may cause errors since the operator API version may not be recognised by Kubernetes.
   `task deploy-resource TARGET=helm_release.elastic_operator`

3. After this, other resources should not have dependency issues, so you can finish the deployment with a full Terraform apply: `task apply`.

3.4 Fission Setup

We use Fission as a Function as a Service (FaaS) framework to run and orchestrate functions with minimal infrastructure setup. In order to manage Fission resources such as packages, functions, routes, and triggers, we use Fission specifications. In what follows we will explain our Fission setup and utilities we set up to help run functions.

3.4.1 Environment

The default Fission Python environment uses a very outdated version of Python and of the packages used by Fission. Therefore, in order to use more a more recent Python version, we created a custom Fission environment for Python. Some changes we made to create this custom environment are:

- Use up-to-date Python version 3.13.
- Use updated versions of the packages used in the Fission environment, such as Flask and Sentry. We dropped the apparently abandoned "bjoern" package and replaced Redis for Valkey as well.

The Docker image for this environment was published to DockerHub, where it is made publicly available. In our Fission specifications we create this environment under the name "python313" and use it to run all our functions.

4 Data Gathering

For this project, we gathered data from several sources. These included several social media sites as well as the GDELT [Leetaru and Schrodt, 2013] events data source. In order to collect data from these sources, we initially assigned one data source to each team member to explore. Responsibility fell on each team member to collect the data and save what we considered relevant to our ElasticSearch database. This worked well for some sources that we were able to get working

4.1 Social Media Data Gathering Strategy

4.1.1 Architecture

In order to gather data from social media sources, we used several Social Media sites that provided APIs for accessing public data. These were Youtube, Mastodon, and Bluesky. We gathered data from these sources continuously using a Fission-based serverless architecture with the following components (See Figure 2):

1. Data gathering functions: These are the actual functions in charge of gathering data from the given sources using their APIs. These functions are in charge of simply gathering the data from the source and filtering out the data that could be relevant to our use cases. These functions focus on getting real-time data from the sources as it comes in rather than batching historical data. When one of these functions gathers a record of data, it is put into a Valkey queue, which will in turn trigger the processing function via the Kubernetes Event Driven Autoscaling Fission component (Keda).

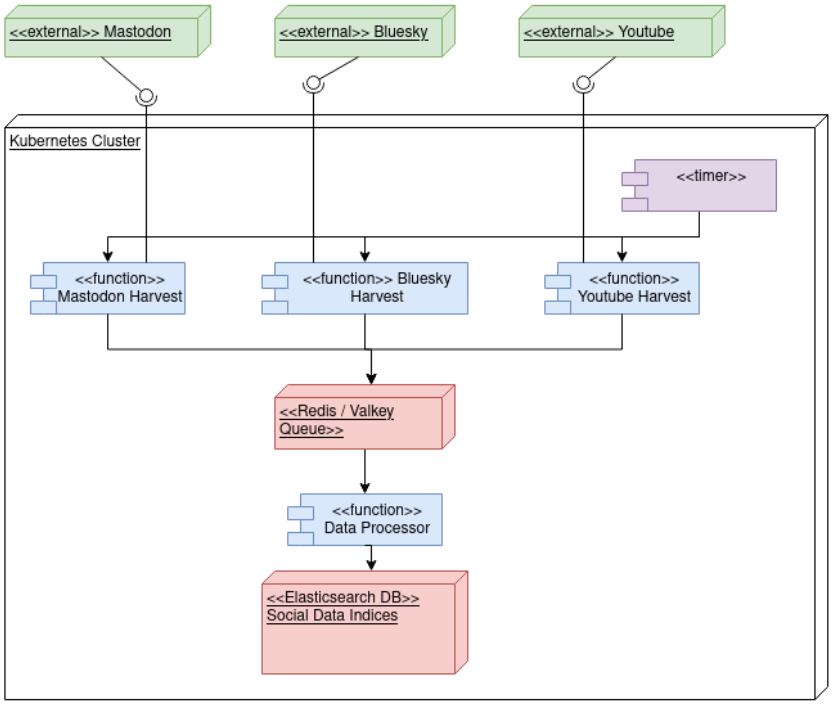


Figure 2: Architecture for Gathering Social Media Data.

2. Valkey: This is a Redis drop-in replacement we used to handle message queues used by Fission and Keda to integrate various functions in our pipeline. In this case, it serves to decouple data gathering from processing.

3. Data processing function: This function is in charge of receiving data from the various social media sources, applying some transformations to the data and inserting it into our ElasticSearch indices. This function is kicked off when records are pushed to the `social-posts-index` queue on Valkey, and will process a single record in each call.

Using this message queue approach has several advantages. First, it decouples data gathering from data processing and insertion, which improves modularity and helps keep the data gathering functions lightweight and focused. Using the message queue also allows us to control the concurrency of the data processing function (which could potentially take some time) and let Valkey absorb spikes in traffic from the social media sources without throttling ingestion or overwhelming the cluster's compute capacity. If the volume of ingested data were to increase dramatically, we could scale Valkey from a single instance to a cluster and scale out the data processing function by increasing its maximum concurrency in its Fission function specification.

4.1.2 Implementation Notes

Within our project, we placed all our harvesting code in a single `harvesters` package both as a Python workspace and as a Fission package. This allows us to share common utilities for all functions such as credential parsing and management, and reduces the overhead of building and deploying multiple Fission packages when applying our specs. The difference between our various Social Media gathering functions is only their entrypoint and what specific submodules of our code they use.

4.2 Social Media Providers

4.2.1 Mastodon

Data Streaming One of the data sources we used was Mastodon. This is a federated network with several distinct servers, many of which have few users. In order to restrict ourselves to data specifically relevant to Australia in some way, we took the following approach:

- Fetch all data from the public timeline of the Australia-focused servers `https://mastodon.au/` and `https://aus.social/`. Here we restricted ourselves to the local timeline of these services. This is to avoid data from federated servers that could come through without being relevant to Australia if we did not set this filter.
- Fetch data from the central *https://mastodon.social* server and all its federated servers. In order to limit ourselves to data relevant to Australia, we fetched only data for a set of hashtags broadly related to Australia.

To harvest this data, we set up Fission functions that would run every 30 seconds for each of the three chosen Mastodon servers. The functions would use Mastodon's streaming functionality (via Server-Side Events) to stream posts as they came in for 30 seconds before shutting down for the next function call to take over.

Backfill Getting historic data from the Mastodon API was quite challenging, as it returned very few results. Because of this, we decided to use the data dump from SPARTAN used during the first assignment. This data certainly gave us a great volume of historical Mastodon posts, but unfortunately it did not overlap with the time range we were interested in, namely around the 2025 election or more recent data.

4.2.2 Bluesky

We designed the Bluesky ingestion component to address two complementary collection needs: **live collection**, which captures new posts continuously as they appear on the network, and **historical recovery**, which backfills posts from before the streaming deployment. This led to two modules: a *streaming harvester*, built around Jetstream and WebSockets, and a *historical collector*, built around authenticated HTTP search queries.

Streaming Harvester For live collection, we use Bluesky Jetstream rather than polling standard REST endpoints. In the implementation, the streaming settings define a Jetstream WebSocket subscription endpoint and restrict the stream to post events in the `app.bsky.feed.post` collection, rather than subscribing to all repository updates. The streamer opens a WebSocket connection, continuously receives JSON messages, validates them into a `JetStreamPost` model, and retains only records whose operations are `create` or `update`. This design is consistent with the official AT Protocol / Bluesky description of Jetstream as a lightweight streaming layer built on top of the firehose, with simple JSON encoding, reduced bandwidth requirements, and filtering by collection or repository [AT Protocol Team, 2024, AT Protocol, 2026b].

We chose Jetstream because it is well suited to continuous real-time ingestion. However, Jetstream is primarily designed for live data rather than backfill, which is why a separate historical collection path is required [AT Protocol Team, 2024, AT Protocol, 2026b].

Streaming Filtering and Deployed Data Flow The streaming pipeline is not a raw mirror of all Bluesky traffic. Instead, we apply relevance filtering before forwarding data downstream. The code defines three levels of Australia-related keyword signals: *strong*, *medium*, and *weak*. A post is accepted if it contains at least one strong keyword, at least one medium keyword, or at least two weak location-related keywords. An optional second-stage topic filter can then be applied if a keyword list is provided at runtime. This allows us to first identify broad Australian relevance and then narrow the stream to a particular policy or social topic when needed.

In the deployed Fission entrypoint, `bluesky_stream()` reads configuration from two Kubernetes secrets, namely `valkey-cfg` and `bluesky-main`. It then opens an asynchronous Valkey client and calls the `stream_to_vk(...)` method of `BlueskyStreamer` for a fixed runtime window. Rather than writing directly to a final analysis index, the function pushes queue messages into Valkey, where each item is formatted as `{"source": "bluesky", "data": ...}`. This queue-based design decouples ingestion from downstream processing, so that indexing, schema normalization, and enrichment can be handled separately.

Historical Collector Because Jetstream is fundamentally a live stream, it cannot recover older posts from before the streaming deployment started. For this reason, we implemented a dedicated historical collector. This module uses the Bluesky HTTP APIs instead of WebSockets. It authenticates with `com.atproto.server.createSession`, obtains a bearer token, and then queries `app.bsky.feed.searchPosts` over date-bounded windows [Bluesky, 2026c, Bluesky, 2026b]. The collector uses app-password-style credentials (`handle` and `app_password`) rather than an interactive OAuth flow, which is appropriate for a backend harvesting process [AT Protocol, 2026a, Bluesky, 2026a, Bluesky, 2026c].

Historical Query and Windowing Strategy The historical collector is search-based. It starts from a default list of Australia-oriented search queries, including broad national terms such as `australia`, `australian`, and `auspol`, as well as more specific political and state-related phrases such as `federal election`, `labor party australia`, `victorian government`, `melbourne victoria`, and `queensland government`. We deliberately separate retrieval from filtering: search queries are used to maximize recall, while the strong, medium, and weak keyword lists are applied afterwards to determine whether a retrieved post is sufficiently relevant to keep. This design improves coverage without making the final retained dataset excessively noisy.

A key part of the historical design is time partitioning. The collector implements a `_three_day_windows(...)` helper, which splits a requested interval into sequential 3-day windows. This reduces overlap and improves coverage compared with searching across a very long period at once. The collector also exposes `page_size`, `max_pages`, and `sort` to control query depth within each window. In addition, it applies an explicit `cutoff_before` timestamp so that historical ingestion only collects posts earlier than the streaming deployment boundary, reducing overlap between live and backfilled data.

Normalization, Deduplication, and Reliability. Each search result is normalized into a Bluesky-compatible structure. A deterministic document ID is then formed from `did` and `rkey`. Before sending a normalized record onward, the collector checks whether the common index (section 5.4), `social-posts-index`, already contains the corresponding `bluesky-{did:rkey}` document ID. If the record is new, it is pushed into the Valkey queue as a source-tagged payload. In this way, the historical collector feeds into the same downstream processing path as the live stream.

To improve robustness, the historical collector supports retries for retryable HTTP status codes such as `429`, `500`, `502`, `503`, and `504`, together with exponential backoff [Bluesky, 2026d].

Schema Compatibility. We define explicit mappings for both `bluesky-idx` and `bluesky-historical-idx`. These mappings preserve the same main Bluesky-specific structure, including `did`, `time_us`, `kind`, and `identity.handle`. They also retain a nested `commit` object with fields such as `operation`, `collection`, `rkey`, and `cid`, together with `record.text` and `record.createdAt`. Both mappings also include keyword arrays for matched filter terms. This schema alignment ensures that historical and live Bluesky records can be handled consistently by downstream processors and analysis workflows.

4.2.3 Youtube

The Youtube data is gathered using Data API v3 [Google Developers, 2026] through the python library `google-api-python-client` as a scheduled Fission function, and follows the same harvester → Valkey → processor pipeline as the other providers. The API key is obtained from the Google Cloud Console (create project, enable *Youtube Data API v3*, generate key under *Credentials*) and stored as the secret `youtube-api-credentials`, exposed at runtime as `YOUTUBE_API_KEY`.

Queries and Customisation. Discovery uses `search.list` driven by a topic-to-queries dictionary defined in `youtube/defaults.py`. Pagination is limited at 30 videos per query, 50 comments per video, and 20 replies per comment.

Records, Validation, and Deduplication. The harvester collects three entities — videos, comments, and replies — each retaining its platform ID, parent references, text, author, like count, and publication and update timestamps; comments and replies are later enriched with a sentiment object containing label and score. Videos with disabled comments return HTTP 403 on `commentThreads.list`, so they are pre-filtered through a single `videos.list(part="statistics")` call that discards any with no comments, and every API call is wrapped in a try/except block so isolated failures do not abort the batch. Furthermore, duplicates are suppressed by using the platform ID as the ElasticSearch document ID (re-ingestion replaces the document), a per-run `seen_ids` set, and a short-lived Valkey deduplication key. Because comments can be edited, the harvester preserves the last-update timestamp and overlapping windows guarantee that edits refresh the same document.

Indices and Valkey Pipeline. Data is stored in three flat indices that allow aggregations at any granularity without joins:

Index	Fields
youtube-videos-idx	video_id, title, description, channel, channel_id, published, topic, query
youtube-comments-idx	comment_id, video_id, video_topic, video_channel, video_published, text, author, like_count, published_at, updated_at, sentiment {label, score}
youtube-replies-idx	reply_id, comment_id, plus the same fields as comments

Table 1: Youtube ElasticSearch indices and their fields.

The harvester does not write to ElasticSearch directly; for each entity it pushes a message into the Valkey list `social-posts-index`:

```
{
    "source": "youtube",
```

```
    "data": {
        "kind": "video" | "comment" | "reply",
        "record": {...}
    }
}
```

A KEDA-triggered processor consumes one message per invocation, routes it by the `kind` field, enriches text with sentiment, and writes the document, decoupling the rate-limited harvester from the heavier transformation workload.

Quota and Analytical Outputs. The API enforces 10000 units/day; `search.list` costs 100 units, the other calls cost 1 each. With the caps above and a twelve-hour trigger, usage stays within quota; if volume grew, mitigations are shrinking the query list, lowering caps, requesting a quota increase, or rotating keys. The resulting dataset supports sentiment trends per topic or channel, engagement analysis (likes, comment-to-reply ratios), ranking of channels and authors, polarity comparison between parties or leaders, keyword co-occurrence between titles and comments, and cross-platform alignment with Mastodon, Bluesky, and GDELT through shared topic and time fields.

4.2.4 Threads

We explored getting data from Threads required us to submit an application to Meta. Since the timeframe didn't allow for that, we instead experimented with gathering data using web scraping. We searched for specific keywords and scrolled to the bottom of the page to load more data, repeating this process until it had loaded all the data it could find. This approach was able to gather some data, but it was not very efficient and was prone to breaking whenever Threads made changes to their frontend. For this reason, we decided to focus our efforts on the other data sources for the rest of the project. Along with the inefficiency of this approach, we also had to be careful to avoid making too many requests in a short period of time, as this could have led to our IP being blocked by Threads. We implemented some basic rate limiting in our scraping code to mitigate this risk, but it was still a concern throughout our testing. Furthermore, the addition of chromium to our package would've made it much heavier and more difficult to deploy, which was another reason we decided to abandon this approach.

4.3 Additional Data Sources

4.3.1 GDELT

The Global Database of Events, Language, and Tone (GDELT) monitors global news media in real-time. Our strategy for gathering this data is split into two parts: a live streaming harvester running on our serverless Fission architecture, and a separate, local script for downloading historical data.

Main Flow of Data. For live data, a Fission timer triggers the GDELT harvester every 15 minutes. It downloads the newest updates, formats them, and pushes them to our `gdelt-index` in ElasticSearch. For historical data (backfilling), we wrote a separate Python script that runs locally and uses multi-processing to download specific days in parallel.

A simpler approach would be to use the same Fission function for both live and historical data. However, serverless functions have strict time limits and memory caps. Trying to process months of historical data on Fission would have caused timeouts and overloaded our cluster. Splitting the architecture required us to write and maintain two separate deployment strategies, which increased development time, but it ensured our live pipeline remained lightweight and fast while our local machine handled the heavy lifting of historical backfills.

Harvester Logic and Cross-Referencing. GDELT data is difficult to work with because it is split across different tables. The **Events** table contains the structural facts (who did what, and where), while the **Global Knowledge Graph (GKG)** table contains the context (emotions, tone, themes, and organizations). Our harvester downloads both tables, finds the matching records using their `CAMEOEVENTIDS` or `SOURCEURLS`, and merges them into a single JSON document.

The easiest method would be to just upload the Events table and ignore the GKG table, or to upload them as separate documents. However, doing so would make our dashboards useless, as we needed both the geographic location (from Events) and the sentiment (from GKG) in the same query. Furthermore, instead of letting ElasticSearch generate random IDs for these documents, we engineered a deterministic ID by combining the `GlobalEventID` and the `SourceURL`. While building this matching logic was complex and required careful dictionary mapping in Python, it made our database idempotent—meaning we can accidentally run the script twice without creating duplicate records.

Column Selection and Data Cleaning. Raw GDELT files contain dozens of columns. We apply a strict `USEFUL_COLUMNS` filter to drop most of them, keeping only core geographic data, actors, and the specific GKG sentiment and theme fields. We also run explicit data-cleaning steps, such as padding timestamps with zeros and converting missing values to `None`.

The straight forward approach is a simple "select all" (dumping the entire raw CSV directly into ElasticSearch). We quickly realized this would bloat our database with empty or irrelevant columns, causing unnecessary storage costs and slow queries. We also faced major bugs with data types: data libraries like Pandas try to automatically guess data types, which caused it to turn 14-digit timestamps (like `20241016000000`) into scientific floats (like `2.02E+13`), destroying our time-series data. Pandas also uses `NaN` for missing data, which causes JSON serializers to crash. Fixing this forced us to write explicit data-sanitization functions, which slowed down early development but ultimately prevented silent data corruption.

Fault Tolerance and Performance Optimization. Before uploading historical data, our script performs a final sweep to remove duplicates and saves a clean CSV backup to the local hard drive. Finally, instead of inserting records one by one, it uses ElasticSearch's `parallel_bulk` API to upload chunks of 1500 documents at a time using multiple threads.

When dealing with massive datasets, uploading row-by-row is incredibly slow and fragile. If the internet drops after uploading 50,000 records, a simple script crashes, forcing you to start over and guess where it left off. By taking the time to implement local CSV backups, exponential retry logic, and bulk multi-threading, we made the ingestion pipeline bulletproof. This added significant complexity to our Python code, but it reduced our historical upload time from days down to hours and ensured we never lost data due to a network hiccup.

4.3.2 Official Data

We also incorporated sources of more official data into our study. Namely, we uploaded the 2025 election results [Australian Electoral Commission, 2025] by uploading the Two-Candidate-Preferred vote counts by polling place. We hoped to use this data as a source for geographic as well as trend information, but we were unable to correlate the social media posts with geographic locations directly, since they had no geolocation data. Perhaps this could be studied across accounts in the future, by using each account's entire post history to try to determine their location and thus related them to the candidates that ran in their area.

In the end this data was still useful and insightful, as it showed the major parties and candidates we could target for further study.

4.4 Data Collection Challenges

Data Source Access We faced several challenges when it came to data collection from these sources. One of the first issues we encountered was access to data sources. We had initially hoped to use high-volume social media sites such as Reddit or Threads, but many platforms have tightened their API access policies, so were unable to connect to them. Instead, we were relegated to somewhat lower volume sites such as Mastodon, Bluesky, and Youtube. In the end, Bluesky proved to be our largest source of social media data by far.

Data Relevance As it stands, the biggest challenge we encountered with our data sources was obtaining a large volume of *relevant* data. That is, data that is relevant to Australia in some way. Once we added filtering for our specific use cases, the data volume can become quite meager. A further complication is that the data sources we had available did not have any geolocation information, such as coordinates, which meant that to determine geographic relevance we relied on keywords in the text or hashtags. This approach is overly restrictive, since posts relevant to Australia or made by people in Australia will rarely mention this fact explicitly.

Using the Mastodon streamers and the Bluesky streamer, we could get large volumes of data coming in, but most of it would not really be relevant to Australia or our use cases. In the end getting this critical mass of data while ensuring it was relevant proved to be our Achilles' heel. Perhaps our approach of filtering strictly at the source of the data was not the most effective for gathering large data volumes, as with somewhat looser filters we could still have gathered plenty of relevant historical data at the cost of some later processing and filtering.

Historical Data Another great challenge was gathering *historical* data from the data sources. Using the Mastodon API to obtain historical statuses was simply a non starter, as the server returned very little data from before the present even with pagination. The SPARTAN data dump gave us a great windfall in terms of volume of historical Mastodon posts, but they were unfortunately outside our desired time range. These posts were mostly from 2024 or earlier, which made them less valuable to our case study. Moreover, even some basic relevance filters like excluding posts not made from Australia-specific servers or mentioning a series of keywords discarded most of this data dump as not relevant. This left us with less than 1% of these posts (though this was still in the hundreds of thousands). Bluesky was much more forthcoming in this regard, and we were able to get much more historical data using their API. This was still not a complete picture with all the volume we could have wanted, but it was still quite useful for our efforts.

5 Data Processing

5.1 Architecture

As we stated in our Data Gathering section 4, we included a processor Fission function to transform and enrich raw data from our social media sources and insert it into ElasticSearch. This function was triggered via Keda when one of the "harvester" functions pushed a record to the processing queue on Valkey.

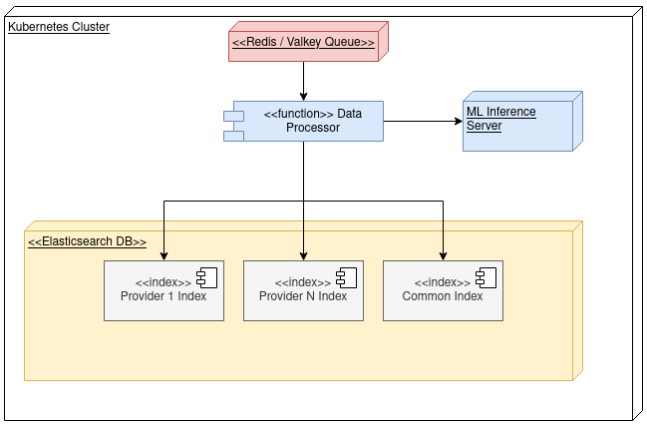


Figure 3: Data processing pipeline architecture

The data obtained from the different sources had different schemas and fields, which made querying data from all of them to study a single problem a challenge. For this reason, we created a common index to store the data from these sources under the same schema, as well as storing the raw-data in provider-specific schemas. This allows us to have both the raw data for validation and further source-specific exploration purposes and the processed data for analysis at the cost of using up more storage space.

As our data of interest was mostly text based, we based our processing around text features and manipulation. ElasticSearch offers native full-text indexing and search capabilities, and we also augmented the text with additional features computed using Machine Learning models. These include text embeddings for semantic search to extend searching capabilities beyond full-text search, sentiment analysis, and emotion classification.

5.2 Machine Learning Inference

In order to use ML models for the data enrichment mentioned in section 5.1, we used open source models from HuggingFace along with their Transformers library. In particular, we used the following:

1. A sentiment analysis model trained on Twitter texts [Camacho-collados et al., 2022].
2. An emotion detection model [Antypas et al., 2023]. This is a classification model that outputs an array of emotions detected in the text.

Both of these Machine Learning models were trained specifically on social media data, namely twitter, which made them well suited for our use case. While the models have not been updated in some time, they can still be quite useful for NLP tasks in resource-constrained environments such as ours.

In order to deploy these models, we converted them to ONNX format and deployed a standalone service to perform inference, which exposed a simple HTTP API. Using ONNX allows us to keep the image more lightweight, instead of using a full-blown ML framework with training and other capabilities we do not need. These models are pre-packaged into the docker image to avoid the overhead of downloading and possibly converting them on container startup at the cost of having a larger image.

We deployed this as a Kubernetes deployment exposed via a service within the cluster, with the configuration definitions written in a Helm chart. Using a helm chart means we can more easily modify configurations for the service such as the number of replicas and the version of the Docker image we are using. We are currently using a single replica to be mindful of limited cluster resources, but this

container can be horizontally scaled by simply adding more replicas, since it is completely stateless. One could also add a Horizontal Pod Autoscaler to the deployment and allow it to scale elastically with usage, but this is currently not enabled.

The reason we decided to deploy this as a standalone service instead of a Fission function is that in principle FaaS functions should be lightweight, and quick to deploy and run. However, using Transformers models that must be loaded into memory can significantly add to the startup time of the service, even with our optimisations, while once they are loaded into memory inference is relatively fast for the chosen models, even without a GPU. Thus, the benefits of deploying this as a standalone persistent service rather than a Fission function outweighed the overhead of managing some additional Kubernetes resources explicitly.

5.3 Language Processing Limitations

While the deployed HuggingFace models proved effective for baseline sentiment and emotion detection, mining social media text presents inherent NLP challenges. Social media discourse is heavily reliant on sarcasm, irony, and culturally specific slang (e.g., Australian colloquialisms), which off-the-shelf models frequently misclassify. For instance, a highly sarcastic post criticizing a political leader's decision might be incorrectly tagged as "positive" by the sentiment model due to the presence of outwardly positive vocabulary.

5.4 Common Index Design

In order to allow easier analysis of the data from all sources, we introduced a common data index. While platform-specific raw indices are useful for preserving the original structure of each data source, they are less convenient for downstream tasks such as topic discovery, sentiment analysis, emotion tagging, and dashboarding across multiple social platforms. To address this, we maintain a platform-neutral document schema in `social-posts-index`, allowing posts from different sources to be accessed by a single query.

The schema design of the common index is intentionally lightweight. Rather than duplicating every nested field from the original source, we store a core set of cross-platform analytical fields. The mapping includes `platform`, `record_type`, `platform_post_id`, `common_id`, `text`, `created_at`, `ingested_at`, `source_url`, and `source_meta`. It also includes a `sentiment` object with positive, neutral, and negative scores, together with a final sentiment label, and a nested `emotion` array with label-score pairs. Notably, `source_meta` is stored as an object with `enabled: false`, which means that we preserve provenance or extra metadata without indexing every internal subfield. This provides a practical compromise between traceability and query performance.

Architecturally, the common index acts as the normalized analytical layer of the system. The streaming harvesters produce source-tagged queue messages. These messages can then be consumed by a downstream processor that standardizes fields into the common schema and enriches them with sentiment and emotion outputs. This means that the common index is not merely an additional storage location; rather, it is the layer where heterogeneous platform records become comparable. The design also supports deterministic identifiers through `common_id` and explicit `platform_post_id` values, both of which are important for deduplication and for tracing a processed document back to its original source record.

From an analytical perspective, the common index enables queries that would be difficult to perform on platform-specific raw schemas alone. Because the common schema consistently exposes post text, timestamps, source platform, and standardized sentiment and emotion outputs, we can more easily perform cross-platform topic exploration, measure changes over time, compare sentiment trajectories by theme, and support a front-end dashboard that does not need to understand each platform's native

data structure. In this sense, the common index serves as the bridge between raw collection and usable social analysis.

We implement this common analytical layer in Elasticsearch as `social-posts-index`, with a compact cross-platform schema that stores normalized post text, temporal metadata, provenance metadata, and sentiment and emotion annotations. This design supports downstream reporting, dashboarding, and comparative analysis across multiple social sources while still retaining enough metadata to trace results back to platform-specific records when necessary.

5.5 Error Handling and Safeguards

We added some error handling mechanisms to the processor in order to avoid some potential pitfalls or failure modes. Two that we considered pressing were:

- Errors in MQTTrigger or in the processing function. We added an error queue to the MQT Trigger configuration for Fission to catch these error messages. These arose either when the processing function threw an exception for some reason, or when the trigger failed to call an available processor function (see 6). These errors could then be re-inserted into the processing queue for another try, or un further inspection discarded. This could be done automatically, and one could add a retry limit, but for our case we audited these errors manually.

- Duplicate inserts. This could arise, for example, if a streaming function spun up before the prevous one had exited. In order to handle such cases, we use Valkey to keep a cache of the IDs of records we have inserted in the last 2 hours to avoid duplicate inserts of fresh data. For historical data, deduplication can be handled in other ways, such as by querying the database directly to see which IDs already exist, but in the real-time processing scenario, caching is a quicker and cheaper alternative to this.

6 Technical Discussion

Now we will describe some of the technical implications and results we found from using our architecture.

6.1 Extensibility

Due to our modular design, it is easy for us to add new data sources to our system, provided we have access to their APIs. Adding a new provider requires implementing the following components:

1. A provider-specific index on ElasticSearch. This index will store raw data from the provider with little transformation for validation purposes.

2. A harvester function. This will be set up to run on a schedule by a Fission timer. This function will gather raw records and push them to the Valkey queue for processing.

3. A processor. This is a Python class or module within the processor function that handles the specifics of this provider's raw data. It must convert it to the common schema and assign a relevant ID. The processor will then perform the needed ML inference and push the data to both the provider index and the common index.

Once these components are implemented and deployed, harvesting from the new provider can begin, and it will fit in seamlessly with the existing pipeline and other data sources. Once the work for this common framework was done, it was a great benefit to have the shared pipeline across sources to handle our analytical queries.

6.2 Scalability

6.2.1 Components

In theory, every component of our architecture is horizontally scalable, which allows for higher volume ingestion, storage and processing.

- ElasticSearch clusters can be scaled horizontally, which would expand the storage capacity and database availability if combined with proper replication. In our setup we were constrained to 2 nodes, but this does not have to be the case if more resources are available.
- Fission functions are serverless and instantiated on the fly, so they can also be scaled horizontally as much as the available hardware allows. It is mostly a matter of increasing the allowed concurrency for each function. We could thus scale the processors and the API functions as much as needed. One can also scale the harvesters, but this depends on the logic for each data source. In our case we were streaming continuously from each provider, so one function sufficed to receive this data. Other cases may benefit from many concurrent harvesters to gather data. One could also use the NewDeploy strategy for Fission, which provides better performance for high-workload functions if sustained high traffic is expected. In our case we made do with the PoolManager deployment strategy.
- Valkey can be scaled horizontally into a cluster as well. This would increase cache capacity for the processing queue and temporary cached values.
- The ML Inference server can also be scaled horizontally if resources are available. With a higher budget, it can also be set to run on a GPU for better performance and inference throughput.

6.2.2 Experimental Results

We got some idea of the challenges that come with scaling our workload during the project. While streaming data with our harvesting functions can produce a constant stream of traffic to the system, it is able to handle it without much issue. The real challenge cam when we performed large-scale backfills of data to the platform. These were done by pushing records directly into the processing queue on Valkey either from a file or a batched API call. In both cases this provided a much more intense and sustained barrage of traffic than we would get from streaming alone. While in theory all our resources can scale horizontally to infinity, we did face a couple challenges and calibration issues to balance the workload.

With the traffic torrent from backfills, the first bottleneck we found was a mismatch between the concurrency configurations of the processor function and the MQT Trigger on Fission. The trigger had a higher concurrency than the function, and so many dispatches failed due to no function pods being available to process data. This was fixed by increasing the concurrency of the processor function as well.

The above was a simple configuration fix, but the major bottleneck we found was the ML inference server. When we increased the concurrency of the processor and the trigger too far, past about 120, the ML server started crashing with some frequency. We tried increasing the replica count of the deployment to 2, but this seemed to worsen the issue, as one of the replicas had trouble getting running. One possible explanation is that since performing inference with transformers is a CPU and memory intensive activity (even in our optimised setup), both replicas might have been competing for limited resources with each other and with the other components running on the cluster.

Thus, we were forced to keep just one replica of the ML server running on the cluster. After some trial and error, we found that we could run 100 concurrent replicas of the processor and trigger with this setup. The ML server continued to crash very occasionally, but considering it was handling dozens of requests per second (2 for each MQT trigger replica, which poll the queue every second), our brave little inference engine help up quite well indeed. With more resources to run the ML server, there is no reason to think we could not greatly increase the throughput of our system.

6.2.3 Fault Tolerance and Single Points of Failure

A critical consideration in our cloud architecture is fault tolerance. By leveraging Kubernetes as our orchestrator, many components are inherently resilient. If a Fission harvester pod or KEDA-triggered processor pod crashes, the Kubernetes control plane detects the failure and automatically spins up a replacement. Furthermore, the use of Valkey as an asynchronous message queue ensures that data is not lost during transient downstream processing outages; messages simply queue up until the processor is back online. The component that proved to be most at risk was the ML Inference server, due to the computational constraints outlined above, but even when this failed, it only caused a temporary outage of processing until a new replica was spun up. The failed jobs were stored in the errors queue and they could simply be rerun once the outage was resolved.

6.2.4 Testing

In order to test the application, we added unit tests to the individual backend components to test the code within them. this helped ensure individual piece worked as expected. We focused on the more crucial pieces of code for testing. Integration tests were more difficult to implement, and we resorted to monitoring the application logs and resources with Kubernetes, as well as ElasticSearch and the processing queues when adding new features to ensure all was working properly. We also added some end-to-end tests for the HTTP API to verify our routing and ElasticSearch queries were in order.

6.3 Tech Stack Discussion

6.3.1 NeCTAR

The Melbourne Research Cloud (MRC) provided a nationally funded academic cloud environment that allowed us to deploy a production-grade kubernetes cluster. The native OpenStack integration meant that Terraform could provision and manage all infrastructure resources, including Kubernetes nodes, load balancers, and storage volumes. This allowed us to focus on developing our application rather than managing infrastructure. Naturally one great advantage of working with NeCTAR in our setting is that it is provided for free for academic uses such as ours.

A possible disadvantage of working with NeCTAR relative to commercial cloud providers is that the latter tend to have many more managed services, such as Function as a Service frameworks. While Fission proved sufficient for our needs, managing it and the intricacies of its environments and deployment was an ever-present overhead for our development. Another limitation is that our available resources were capped from the start, while in a commercial provider we could have added more nodes if we so desired. Moreover, we did not have access to more powerful compute instances such as instances equipped with GPUs. Such instances would greatly expand the possibilities of what we could accomplish in terms of data enrichment and processing with ML models, for instance.

6.3.2 Kubernetes

Kubernetes itself proved to be a very useful tool, without which we could not have achieved what we did given the time constraints. While there is quite a learning curve when first getting started, once it is up and running, it can greatly reduce the overhead of administering a running complex system.

With just a few shell commands we could deploy an entire ElasticSearch database cluster, and barring disasters, managing it is straightforward compared to setting up the cluster from scratch on the bare VMs. This is also a huge advantage of containerised applications, which allow quick deployments in this environment while greatly reducing the management overhead.

Developing with Kubernetes as an orchestrator also has the indirect effect of making our application highly portable, since Kubernetes has become a widespread industry standard. We could just as easily deploy our application on NeCTAR as on a commercial cloud provider or even an on-premises environment provided they have a Kubernetes cluster set up. This portability would be much more difficult to achieve with a commercial provider's specific service offerings for container orchestration, but in exchange for this portability we pay the cost of management overhead of Kubernetes services and a steeper learning curve.

6.3.3 Fission

Fission proved to be a very useful resource for deploying our FaaS components. It provides the automatic management of infrastructure and scaling that one would expect from a FaaS service. This allows one to focus more on the code than the infrastructure requirements, provided one is developing with components that lend themselves well to the FaaS mode. For example, in our use case, the processor and data harvesters are stateless, ephemeral functions that each process a single request, and thus they are the ideal use case for FaaS. This framework also allows us to more easily deal with bursts of traffic such as a surge in activity on a social network or a backfill operation. The use of a Kubernetes-like model of YAML specifications to manage resources also makes it integrate well with the Kubernetes ecosystem we used, and provide a single format for describing resources.

This being said, Fission in particular did have some limitations. For example, we were forced to create and deploy our own Fission environment in order to us updated versions of Python and our library dependencies. The Python environment is also somewhat constrained by its use of flask as an overall wrapper needed to perform all function calls. One could in theory substitute this for another framework, but this would take considerable effort, which was not a luxury we had.

Despite its few shortcomings, Fission was still a valuable resource, as it did indeed reduce the overhead of deploying and integrating our code.

6.3.4 ElasticSearch

ElasticSearch is a great tool for this sort of use case. Its distributed architecture makes it ideal for scaling to large volumes of data, and its document-oriented structure makes it easy to deal with the kind of semi-structured data typical of social media APIs.

ElasticSearch's powerful query engine also allows one to make expressive queries and time series aggregations. The text search features of the engine are also very powerful and of great value for filtering the noisy text that we encountered. One downside of ElasticSearch, however, is that its query

language (DSL) is quite cumbersome and hard to learn. Its other query languages, such as ES|QL, are much more intuitive, but they sill lack the full feature set of the DSL language.

7 Results and Analysis

Our analytics frontend, we first selected Prime Minister Anthony Albanese and examined the weekly average sentiment trend (see Figure 4). In late March 2025, the Australian federal election was officially called for 3 May, marked by the dashed vertical line in the chart. An interesting pattern appears in the pre-election period. In the weeks leading up to election day, Albanese's sentiment series shows a temporary improvement: negative sentiment becomes less dominant, while positive sentiment rises and at some points overtakes negative sentiment. Although the chart alone cannot establish a direct causal relationship, it does suggest that the pre-election period was associated with a more favourable online sentiment pattern for Albanese. Notably, Albanese and the Labor government ultimately won re-election, making this shift in sentiment particularly interesting in hindsight.

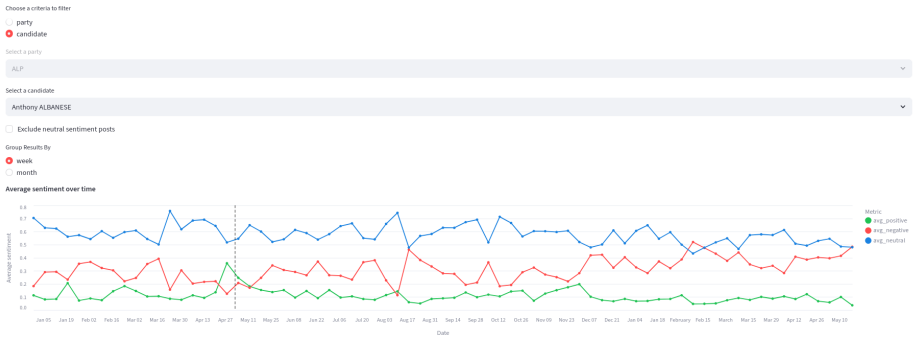


Figure 4: Weekly Average Sentiment for Anthony Albanese.

A similar pattern can also be observed when we switch the selection from Albanese to his party, the Australian Labor Party (ALP). The ALP chart (Figure 5) shows a comparable shift over the same period, with positive sentiment strengthening and negative sentiment easing relative to earlier weeks. This parallel movement across both the candidate-level and party-level charts suggests that the change was not limited to Albanese as an individual figure, but was reflected more broadly in Labor's overall online sentiment during the campaign period. The fact that both Albanese and the ALP later secured victory in the election further reinforces the significance of this shared pre-election sentiment pattern.

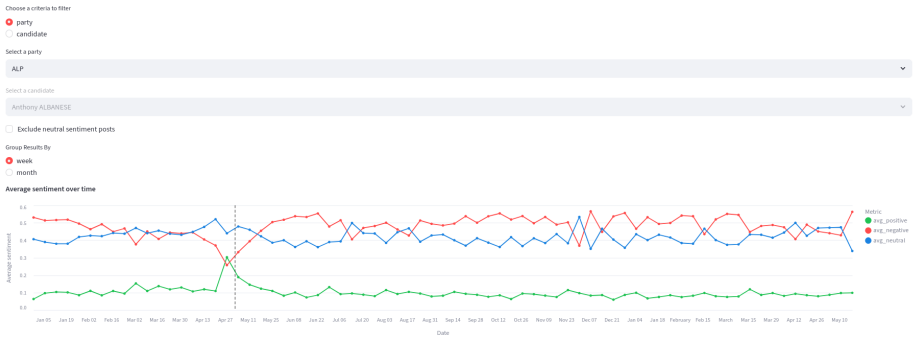


Figure 5: Weekly Average Sentiment for the Australian Labor Party.

To provide a broader context to these social media trends, we cross-referenced our findings with data harvested from the GDELT project, which tracks global news media sentiment and event locations. When applying the same temporal analysis to the Australian Labor Party's news coverage (Figure 6) and Anthony Albanese's individual coverage (Figure 7), a striking correlation emerges. In both GDELT timelines, the period immediately preceding the May election (marked by the dashed vertical line) shows sentiment peaking towards 0.0 (Factual/Neutral). This confirms that the pre-election "bump" in positive sentiment observed on social media platforms was mirrored by a stabilization and improvement in mainstream media coverage.



Figure 6: GDELT Weekly Average Sentiment and Article Volume for the Australian Labor Party.



Figure 7: GDELT Weekly Average Sentiment and Article Volume for Anthony Albanese.

However, mapping the GDELT event data reveals a significant divergence in how the Prime Minister is treated in the media compared to his party, particularly in the post-election landscape. The geographic sentiment map for the Australian Labor Party (Figure 8) displays a largely stable, neutral-to-positive distribution (green and yellow nodes) clustered along the eastern seaboard and major metropolitan areas.

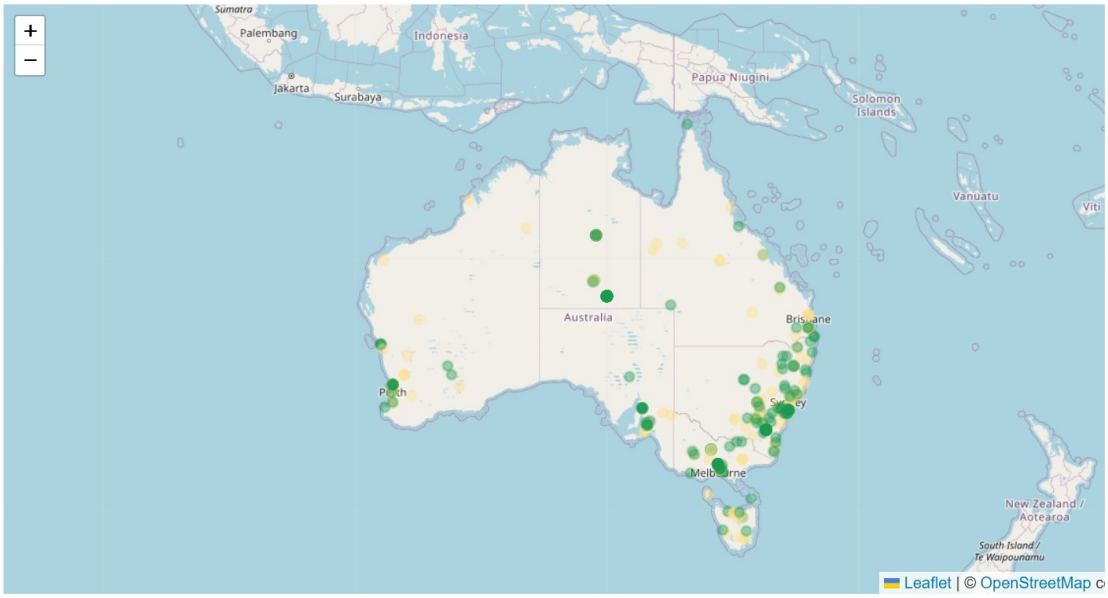


Figure 8: GDELT Geographic Sentiment Distribution for the Australian Labor Party.

Conversely, the geographic map for Anthony Albanese (Figure 9) is distinctly more polarized, featuring highly visible clusters of negative sentiment (red nodes) surrounding major capital cities such as Perth, Melbourne, Sydney, and Brisbane. This geographic polarization aligns with the severe downward spikes into "Crisis/Scandal" territory (sentiment < -3.5) observed in Albanese's timeline during late 2025 and early 2026. This suggests that while social media and news media both reflect a collective party lift during election campaigns, the party leader individually absorbs the vast majority of negative media scrutiny and localized political tension during the actual governance period.

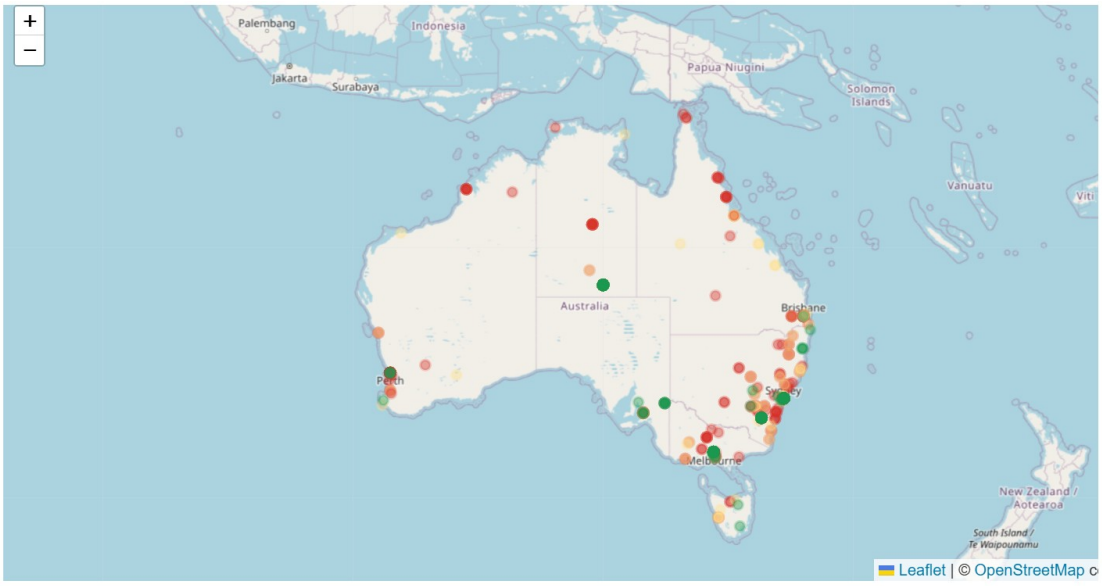


Figure 9: GDELT Geographic Sentiment Distribution for Anthony Albanese.

8 Conclusion & Limitations

This project successfully demonstrated the design, deployment, and operation of a scalable, cloud-native social media analytics platform on the NeCTAR Research Cloud. By leveraging Kubernetes for robust orchestration, Fission for serverless data harvesting, and ElasticSearch for powerful data storage and querying, we established a pipeline capable of ingesting and enriching disparate data streams from BlueSky, Mastodon, YouTube, and the GDELT project.

The architecture's decoupled nature, facilitated by the Valkey message queue, proved highly effective at absorbing traffic spikes and ensuring fault tolerance during heavy data backfills. Our deployment of ONNX-optimised machine learning models provided critical sentiment and emotion enrichment, proving that complex NLP tasks can be achieved even within strictly resource-constrained, CPU-only academic cloud environments.

However, several limitations remain. Gathering high volumes of strictly relevant, geo-located Australian data proved difficult due to platform API restrictions and the lack of explicit location metadata on modern social networks. Consequently, our reliance on keyword-based filtering required navigating a constant trade-off between precision and recall. Additionally, while our ML pipeline effectively established baseline sentiment trends, it struggles with the nuances of sarcasm, irony, and Australian colloquialisms. Finally, due to absolute hardware limits on our NeCTAR cluster, some components—such as our ML inference pod—operate as single points of failure, preventing true high availability.

Despite these limitations, the system successfully meets the core objectives of the assignment. It provides a flexible, dynamically scalable framework that researchers can use to capture the essence of public discourse and visualize shifting political attitudes via interactive Jupyter Notebook analytics. Future work could improve upon this foundation by fine-tuning the NLP models for Australian political slang and migrating the database into a fully replicated, multi-node cluster to ensure complete fault tolerance.

Declaration of Generative AI Usage

During the preparation of this project, the team utilised Generative AI (Google Gemini Flash & Pro 3.1 and Claude Sonnet 4.6) to assist with proofreading, structuring the LaTeX report, and refining academic language. These specific models were selected due to their strong proficiency in processing complex technical context and producing highly structured, academic prose. All system architecture, code, analytical scenarios, and all concepts represent the original work of the student team.

References

[AIReD, 2025] AIReD (2025). Aired website. https://www.aio.eresearch.unimelb.edu.au/.

[Antypas et al., 2023] Antypas, D., Ushio, A., Barbieri, F., Neves, L., Rezaee, K., Espinosa-Anke, L., Pei, J., and Camacho-Collados, J. (2023). Supertweeteval: A challenging, unified and heterogeneous benchmark for social media nlp research. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.

[AT Protocol, 2026a] AT Protocol (2026a). Auth. AT Protocol Docs. Accessed: 2026-05-19.

[AT Protocol, 2026b] AT Protocol (2026b). Streaming data. AT Protocol Docs. Accessed: 2026-05-19.

[AT Protocol Team, 2024] AT Protocol Team (2024). Introducing Jetstream. AT Protocol. Accessed: 2026-05-19.

[Australian Electoral Commission, 2025] Australian Electoral Commission (2025). 2025 federal election results: House of representatives downloads.

[Bluesky, 2026a] Bluesky (2026a). Api hosts and auth. Bluesky Documentation. Accessed: 2026-05-19.

[Bluesky, 2026b] Bluesky (2026b). app.bsky.feed.searchposts. Bluesky Documentation. Accessed: 2026-05-19.

[Bluesky, 2026c] Bluesky (2026c). com.atproto.server.createsession. Bluesky Documentation. Accessed: 2026-05-19.

[Bluesky, 2026d] Bluesky (2026d). Rate limits. Bluesky Documentation. Accessed: 2026-05-19.

[Camacho-collados et al., 2022] Camacho-collados, J., Rezaee, K., Riahi, T., Ushio, A., Loureiro, D., Antypas, D., Boisson, J., Espinosa Anke, L., Liu, F., Martínez Cámara, E., et al. (2022). TweetNLP: Cutting-edge natural language processing for social media. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–49, Abu Dhabi, UAE. Association for Computational Linguistics.

[Elasticsearch B. V., 2018] Elasticsearch B. V. (2018). *Elasticsearch*, volume 6. Elastic B.V. software, version 6(1).

[Fiber, 2026] Fiber (2026). Social media statistics 2026: A look at Facebook, Instagram, YouTube and TikTok for Australian businesses. `https://www.fiber.com.au/blog/social-media-statistics-2026-a-look-at-facebook-and-instagram-for-australian-businesses/`. Accessed: 2026-05-19.

[Fission Contributors, 2026] Fission Contributors (2026). Fission.io – Documentation for Fission. `https://fission.io/docs/`. Accessed: 2026-05-19.

[Google Developers, 2026] Google Developers (2026). YouTube Data API v3 – reference. YouTube Data API Documentation. Accessed: 2026-05-19.

[Jin et al., 2020] Jin, T., Bercea, G. T., Le, T. D., Chen, T., Su, G., Imai, H., and Eichenberger, A. E. (2020). Compiling onnx neural network models using mlir. *arXiv preprint arXiv:2008.08272*.

[Leetaru and Schrodt, 2013] Leetaru, K. and Schrodt, P. A. (2013). Gdelt: Global data on events, location, and tone, 1979–2012. In *ISA Annual Convention*, volume 2, pages 1–49.

[Malla et al., 2020] Malla, A. A., Shinde, S., and Mulla, M. M. (2020). Self-managed block storage scheduling for openstack-based cloud. *Procedia Computer Science*, 171:1439–1444.

[Molleti, 2022] Molleti, R. (2022). Kubernetes advanced auto scaling techniques. *Journal of Mathematical & Computer Applications*, 1(4):2–4. SRC/JMCA-126.

[Sivarajah et al., 2017] Sivarajah, U., Kamal, M. M., Irani, Z., and Weerakkody, V. (2017). Critical analysis of big data challenges and analytical methods. *Journal of Business Research*, 70:263–286.

[Valtanen, 2023] Valtanen, V. (2023). IMPROVING THE MAINTAINABILITY AND DEVELOPER EXPERIENCE OF TERRAFORM CODE. *arXiv preprint*.

[Vassiliadis, 2009] Vassiliadis, P. (2009). A survey of extract–transform–load technology. *International Journal of Data Warehousing and Mining (IJDWM)*, 5(3):1–27.